

Raspberry pi as master

27 May 2026 15:12

raspi-config internally:

- modifies Linux configuration files
- enables kernel drivers/modules
- changes boot configuration

-> on Raspberry Pi Linux:

User applications cannot directly access hardware registers
Instead:

Application

↓

Linux SPI Driver

↓

SPI Hardware

So your C program talks to:

/dev/spidev0.0
which is a:

device file

-> SPI configuration in Linux uses:

ioctl()

This is VERY important in Linux drivers.

ioctl means:

Input Output Control

Used to:

- configure drivers
- send control commands to kernel drivers.

Why are we using uint8_t tx_buf[1] instead of just uint8_t data ?

Because Linux SPI driver expects a memory buffer address (pointer) not just a single value.

-> spi config happens through struct
spi_ioc_transfer spi;

spi.speed_hz controls frequency of SPI clock generated by master which determines how fast bits are shifted.

ioctl means:

Input Output Control

Used in Linux to:

- configure drivers
- send commands to drivers
- control hardware through kernel drivers.

General Format : ioctl(file_descriptor, command, argument)

ioctl(fd, SPI_IOC_MESSAGE(1), &spi);
means: Send SPI transfer request to Linux SPI driver

Communication Flow

Raspberry Pi 4 Model B:

- generates SCLK
- controls NSS
- sends MOSI data

STM32:

- waits for NSS active
- receives clock from master
- shifts incoming data into SPI data register

1. Libraries and their uses

stdio.h → printf()

stdint.h → uint8_t, fixed-size data types

unistd.h → sleep(), close()

fcntl.h → open(), O_RDWR

sys/ioctl.h → ioctl()

linux/spi/spidev.h → SPI structures/macros for Linux SPI driver

2. Linux SPI Driver

Applications on Raspberry Pi 4 Model B do not directly access SPI registers.

Program communicates with Linux SPI driver, which controls SPI hardware.

3. SPI Device File

SPI hardware appears as device file:

/dev/spidev0.0

Linux treats hardware devices like files.

4. File Descriptor (fd)

fd stores the handle returned by open().

Used to identify and access the opened SPI device.

5. Opening SPI Device

open("/dev/spidev0.0", O_RDWR) connects program to Linux SPI driver.

O_RDWR is used because SPI transmits and receives simultaneously.

6. SPI Buffers

tx_buf stores data to transmit and rx_buf stores received data.

Linux SPI driver works with memory buffers, not standalone variables.

7. SPI Transfer Structure

spi_ioc_transfer stores complete SPI transaction information.

Contains buffer addresses, transfer length, speed, bits per word, etc.

8. Passing Buffer Addresses

TX and RX buffer addresses are passed to Linux SPI driver.

Driver reads/writes SPI data directly from/to memory buffers.

9. Transfer Length

spi.len = 1 means transfer 1 byte.

Linux generates 8 SPI clock pulses for this transfer.

10. SPI Clock Speed

spi.speed_hz sets SPI clock frequency generated by master.

Clock determines how fast bits are shifted.

11. Bits Per Word

spi.bits_per_word = 8 means one SPI frame contains 8 bits.

8-bit SPI communication is the most common mode.

12. Transfer Delay

spi.delay_usecs sets delay between SPI transfers.

Used when slave devices require processing time.

13. Starting SPI Communication

ioctl() sends SPI transfer request to Linux SPI driver.

This is the line that actually starts SPI communication.

14. SPI Transfer Flow

Linux SPI driver reads TX buffer, generates clock, shifts MOSI/MISO data, and updates RX buffer.

Received data finally appears in rx_buf.

15. Printing Received Data

printf() displays received SPI data stored in RX buffer.

Useful for debugging and monitoring communication.

16. Infinite Loop

while(1) continuously repeats SPI communication.

Without loop, SPI transfer happens only once.

17. sleep(1)

Adds delay between consecutive SPI transfers.

Prevents continuous high-speed looping and reduces CPU usage.

18. close(fd)

Closes SPI device and releases Linux driver resources.

Usually executed when program exits.

Stm32 as slave

27 May 2026 17:37

1. Enable GPIOA clock

- Required to configure PA4, PA5, PA6, PA7.
- Needed to access MODER, AFR and PUPDR registers.

2. Enable SPI1 clock

- Required to power the SPI peripheral.
- Enables operation of shift register, DR, SR, RXNE, TXE, etc.
- In slave mode, it does NOT generate SCK.
- SCK is supplied externally by the master.

Communication Flow

Raspberry Pi 4 Model B:

- generates SCLK
- controls NSS
- sends MOSI data

STM32:

- waits for NSS active
- receives clock from master
- shifts incoming data into SPI data register

Hardware connections:

STM32		Raspberry pi
PA5 (SCK)	--->	Pin 23 (SCK) GPIO 11
PA6 (MISO)	--->	Pin 21 (MISO) GPIO 9
PA7 (MOSI)	--->	Pin 19 (MOSI) GPIO 10
PA4 (NSS)	--->	Pin 24 (NSS) GPIO 8
GND	--->	Pin 6 (GND)

Why does Pi receive 0xFF even when STM32 is not running?

SPI is full duplex.

For every byte transmitted:

Master TX ---> Slave

Master RX <--- Slave

The Pi always samples the MISO line while transmitting.

If nobody is actively driving MISO, the line is typically pulled high (or floats high), so the Pi reads:

11111111 = 0xFF

Therefore:

TX = 0x55

RX = 0xFF

can happen when:

- STM32 is not powered
- STM32 is held in reset
- STM32 code is not running
- MISO wire is disconnected
- SPI slave is not configured